The Open Source Audit Course: Open Source Software (OSS NGM

Name: Anmol Bhati

Registration Number: 24MIM10170

Date of Submission: March 30, 2026

Chosen Software: VLC Media Player

Part A – Origin and philosophy (Unit 1 & 2)

Part A1 - The problem solved -

VLC media player was not started as a compony or a big startup and anything like that . It actually goes back to 1996, and it was just a group of students in Paris. From what I understand, they were trying to stream videos over their university network, but the network itself wasn't that strong, and the software that was available at that were not that much better.

So instead of trying to fix the network or wait for better tools, they kind of went in a different direction—they decided to build something themselves. That's how VideoLAN started. Initially, it is very basic, just a server to send the video and a client to play it.

Not a big thing just solving a simple problem .

But then, over time, it turned out that what they built was actually was effective. Not perfect, but definitely useful beyond just their university. And I think around 2001, they made it open source, which was honestly a very smart move, even if they didn't fully realize the scale of what would happen next.

Because once it was open, developers from different places started using and helping to improve it4. Instead of one team trying to support every possible video format—which is almost impossible—it became this collaborative thing. People added support, fixed issues, improved performance… and it kept growing in a very organic way.

So I guess the philosophy of VLC is not really about being the most advanced or the most polished media player. It's more about being reliable and open. It just works, most of the time, without asking too many questions. And that's probably why it became so widely used.

It’s kind of interesting actually—something that started as a small workaround for a university problem ended up becoming a global tool, not because it was pushed hard, but because it was genuinely useful.

Part - A2 The lisence –

When it comes to VLC, one thing that quietly defines it—but most people don’t really notice—is its license.

VLC is released under the GNU General Public License, or GPL. Now, at first glance, this just sounds like a technical or legal detail, something you would normally ignore. But if you look a little deeper, this is actually where the real philosophy of VLC sits.

So the basic idea of this license is that the software is “free”—but not just free as in cost. It’s free in terms of freedom. You can use it, study it, modify it, and even share it with others. There’s no restriction on access. But there’s one important condition that changes everything: if you modify VLC and distribute your version, you must also keep it open source under the same license.

And this one rule has a very strong consequence.

It basically prevents anyone from taking VLC, improving it privately, and then turning it into a closed, profit-controlled product. You can build on top of it, but you cannot lock it away. In a way, the license enforces a kind of fairness—it ensures that whatever value is added stays available to everyone.

Now if you think about it, this directly connects to how VLC evolved. Earlier, we saw that developers from all over the world contributed to it. That only works when there is trust. And this license creates that trust. Everyone knows that their contribution won’t be taken and hidden behind some company’s wall later.

But there’s also another side to this.

Because of this license, VLC never really became a commercial product in the traditional sense. You won't see ads, subscriptions, or aggressive monetization strategies. Not because the developers lack capability, but because the structure itself doesn't support that direction easily. It naturally pushes the project towards utility rather than profit.

And this is where it becomes interesting from a philosophical point of view.

Most software today is built with control in mind—control over users, control over data, control over distribution. VLC, on the other hand, is built with the idea of shared ownership. No single entity dominates it. It just exists as a tool that anyone can rely on.

So in a way, the license is not just a legal framework—it's like an invisible boundary that shapes the behavior of the entire project. It decides what VLC can become and, more importantly, what it can never become.

And maybe that's why VLC feels different. It's not trying to pull you into an ecosystem or extract something from you. It just does its job and stays out of your way.

A small technical decision, but with very long-term consequences.

Part - A3 The ethics –

When we talk about VLC, beyond its technology and license, there's also a quiet layer of ethics behind it—which, again, most people don't actively notice, but they experience it.

VLC is developed by the VideoLAN project, and their approach has always been very user-first. They don't treat users as data points or revenue sources. There's no tracking, no ads, no hidden data collection running in the background. In a time where most software tries to extract value from users, VLC almost does the opposite—it minimizes interference.

And that is a ethics choice.

Because technically, they could add ads, collect usage data, or push premium features. But they choose not to. The focus stays on providing a clean, reliable tool without manipulating the user's behavior. There's no attempt to keep you hooked or dependent—it just works when you need it.

Another ethical aspect is accessibility. VLC is designed to run on almost any system, even low-end devices. This might not sound like a big deal, but it actually is. It means people who don't have high-end hardware or fast internet can still access media without limitations. So in a way, it reduces the gap instead of increasing it.

Also, because it's open source, everything is transparent. Anyone can see and observe the code, verify what it's doing, and even suggest improvements. That creates a level assurance that most closed software simply doesn't have.

So overall, the ethics of VLC media player have been said aloud—they're fitted in its design choices. It respects the user's privacy, avoids unnecessary control, and stays focused on utility rather than going the wrong way around .

And maybe that's why people trust it without even thinking about it.

Part B – Linux footprinting (Unit 3)

Part B1 - Installations and directions

For this part, I was basically looking at how VLC operates on a Debian-based Linux system.

The installation part is actually very straightforward. You don't really need to go into anything complicated like compiling it from source or manually setting things up—unless you specifically want that level of control. In most cases, you just use the standard package manager. So you run something like sudo apt install vlc, and it handles everything for you in the background.

Now once it’s installed, the main executable file—basically the actual program you run—will be placed in the /usr/bin/vlc directory. This is the standard applications in Linux, so nothing unusual there.

VLC also creates configuration files, but it doesn’t put them in some global system directory. Instead, it keeps them user-friendly and certain. So for each user, the settings are usually stored inside their home directory, under something like .config/vlc. And since it’s a hidden directory, you don’t normally see it unless you’re looking for it.

This separation actually is actually a good thing . The main program stays at the system level, while the personal settings stay with the user. So different users on the same system can have completely different VLC configurations without sharing nothing with each other.

It’s a small detail, but it shows how cleanly things are structured.

```
Package: vlc
Version: 3.0.20-3build6
Priority: optional
Section: universe/graphics
Origin: Ubuntu
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Original-Maintainer: Debian Multimedia Maintainers <debian-multimedia@lists.debian.org>
Bugs: https://bugs.launchpad.net/ubuntu/+filebug
Installed-Size: 200 kB
Provides: mp3-decoder
Depends: vlc-bin (= 3.0.20-3build6), vlc-plugin-base (= 3.0.20-3build6), vlc-plugin-qt (= 3.0.20-3build6), vlc-plugin-video-output (= 3.0.20-3buil
d6)
Recommends: vlc-l10n (= 3.0.20-3build6), vlc-plugin-access-extra (= 3.0.20-3build6), vlc-plugin-notify (= 3.0.20-3build6), vlc-plugin-samba (= 3.0
.20-3build6), vlc-plugin-skins2 (= 3.0.20-3build6), vlc-plugin-video-splitter (= 3.0.20-3build6), vlc-plugin-visualization (= 3.0.20-3build6)
Suggests: vlc-plugin-fluidsynth (= 3.0.20-3build6), vlc-plugin-jack (= 3.0.20-3build6), vlc-plugin-pipewire, vlc-plugin-svg (= 3.0.20-3build6)
Homepage: https://www.videolan.org/vlc/
Task: lubuntu-desktop, ubuntustudio-video, ubuntu-unity-desktop
Download-Size: 36.6 kB
APT-Sources: http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages
Description: multimedia player and streamer
```

To make sure I had the right repository info, I pulled up the package details directly from the terminal. Here’s what that looks like.

```
/usr/bin/vlc
```

Here’s the quick terminal check I ran just to confirm the exact path of the main file.

Part - B2 - User security and services –

Now when I looked at VLC from a security perspective, one thing that really stood out was how it handles user privileges.

VLC actually refuses to run as the root user by default. And this is not just a random restriction—it's a very deliberate security design.

Because if you think about it, a media player is constantly decoding complex file formats—video, audio, subtitles—all of which involve a lot of low-level processing. Now if someone embeds malicious code inside a media file, it could exploit something like a buffer overflow when VLC tries to decode it.

Now here's the critical part.

If VLC were running as root, that malicious code would basically get full control over the system. Not just the media player—it could compromise the entire operating system. But since VLC runs only as a standard user, the impact is contained within that user's environment. It creates a kind of boundary.

So instead of maximizing power, VLC is intentionally limiting itself to reduce risk. That's a strong security mindset.

Then there's another aspect—how it behaves in terms of services.

VLC is not a background service. It's not like a web server such as Apache HTTP Server, where you manage it using commands like systemctl start or keep it running continuously.

VLC only runs when the user explicitly starts it—either by clicking it or running the command. There's no always-on process in the background.

And that matters, because anything running continuously becomes a potential attack surface. By not existing until it's needed, VLC reduces unnecessary exposure.

So overall, both choices—refusing root access and avoiding background service behavior—show a very practical approach to security. It's not trying to be overpowered, it's trying to be safe by design.

```
anmolbhati858@cloudshell:~$ ls -l /usr/bin/vlc
-rwxr-xr-x 1 root root 0 Mar 31 10:15 /usr/bin/vlc
```

Part - B3 - The update Model –

When it comes to keeping VLC secure over time, the update model is quite structured—but a bit indirect.

VLC itself is developed by the VideoLAN community. So whenever a bug or a security issue is found, they fix it at the source level and release an updated version of the code.

But on a Debian-based system, that update doesn't reach you directly from them.

Instead, it goes through the Linux distribution's package maintainers—like the teams behind Debian or Ubuntu. These maintainers take the updated VLC source code, package it properly so it fits the system's standards, test it for stability, and then push it into the official repositories.

You are not depending on VLC developers—you're also relying on your distribution's maintainers to verify and deliver those updates in a stable way.

And from the user's side, the process is very simple.

You just run something like apt upgrade, and the system automatically pulls in the latest secure version of VLC along with other updates. There's no need to manually download patches or track vulnerabilities yourself.

So even though there's a lot happening behind the scenes, for the user it reduces down to one clean action—keeping the entire system, including VLC, up to date.

Part C – The FOSS Ecosystem (Unit 3&4)

Part C1 - Dependencies and the Shoulders of Giants –

When I looked deeper into VLC, one thing became very clear—no open-source project really stands alone, and VLC is a perfect example of that.

It actually relies on a whole ecosystem of other tools to function properly. The biggest dependency here is FFmpeg. So VLC itself doesn't try to decode every single video or audio format from scratch. That would be extremely inefficient and honestly unrealistic.

Instead, it uses FFmpeg's libraries to handle the heavy work—things like decoding MP4, MKV, and all the complex audio-video formats. You can think of FFmpeg as the engine that does the deep processing, while VLC sits on top and uses it.

Then there's the graphical side of things.

For the user interface—the buttons, menus, sliders, everything you actually interact with—VLC uses another open-source framework called Qt. So instead of building a UI system from scratch, it relies on Qt to provide a structured and consistent interface across different platforms.

And when you step back and look at this, it tells you something important.

VLC is not trying to reinvent everything. It's building on top of already strong, well-developed tools. It's like standing on the shoulders of giants—using existing expertise and focusing only on integrating it properly.

Because if they had tried to build everything from zero—decoding engines, UI frameworks, format support—it would have taken an enormous amount of time, probably decades.

So in a way, VLC's strength is not just in what it builds, but in how intelligently it uses what already exists.

Part C2 - Web connection and the Stack –

When placing VLC in the broader internet ecosystem, it's important to understand where it doesn't fit first.

It's not part of the traditional LAMP stack. So it's not a web server like Apache HTTP Server, and it's not handling databases like MySQL. It doesn't sit on the backend of websites.

But that doesn't mean it's disconnected from the web.

VLC was originally built with streaming in mind, so it works closely with network protocols like HTTP and RTSP. These protocols allow it to fetch and play media directly over networks, not just from local files.

And earlier, it even had browser integration. Through plugins for browsers like Mozilla Firefox, users could play videos directly inside the browser without relying on proprietary tools like Adobe Flash Player.

So even though VLC isn't part of the web stack structurally, it still connects deeply with how media flows across the internet.

Part C3 - Community and governance –

When it comes to who actually runs VLC, it's not a company in the traditional sense.

It's governed by VideoLAN, which is a registered non-profit based in France. So there is a formal structure, but it's not profit-driven or controlled by corporate interests.

But honestly, the more interesting part is the community behind it.

VLC is largely volunteer-driven. Developers from different parts of the world contribute to it, not because they're assigned tasks by a manager, but because they choose to. And the way they coordinate is quite different from modern corporate workflows.

They use more traditional open-source communication methods—things like IRC channels for real-time discussions, developer mailing lists for detailed conversations, and community forums where issues and ideas are shared openly.

There's no central "boss" deciding direction in a strict top-down way. Instead, decisions evolve through discussion, contribution, and consensus. People fix bugs, add support for new formats, and improve performance simply because they want the project to get better.

So in a way, VLC is not just software—it's a collaborative system where structure exists, but control is distributed.

And that's what keeps it alive and evolving even without a traditional hierarchy.

Part D – Open source vs Propeitry -

Dimension	Open Source (VLC)	Proprietary Alternative (Windows Media Player)
Cost	100% Free, forever. No ads, no hidden paywalls.	Included with Windows, but you are paying for the Windows OS license to use it.
Security (who can audit the code?)	Fully transparent. Thousands of independent developers can inspect the code for vulnerabilities.	Closed source. You must trust Microsoft's internal team to find and patch security flaws.
Support and reliability	Driven by a passionate community, forums, and mailing lists. Patches are often pushed rapidly.	Corporate support. Highly reliable, but updates are tied to Microsoft's massive corporate release schedule.
Freedom to modify	Total freedom. Anyone can fork the code, write custom plugins, or change the UI.	Zero freedom. Modifying the software violates the End User License Agreement (EULA).

Based on this comparison, I would absolutely recommend VLC for real-world deployment. It completely removes vendor lock-in and doesn't force users to buy an OS just to play media. As for contributing, even though I'm learning AI and not writing C++ video decoders right now, I would definitely be open to contributing by helping with documentation or testing updates on different Linux systems.

Part E – The Shell Scripts Demonstration

Script 1 - System Identity Report –

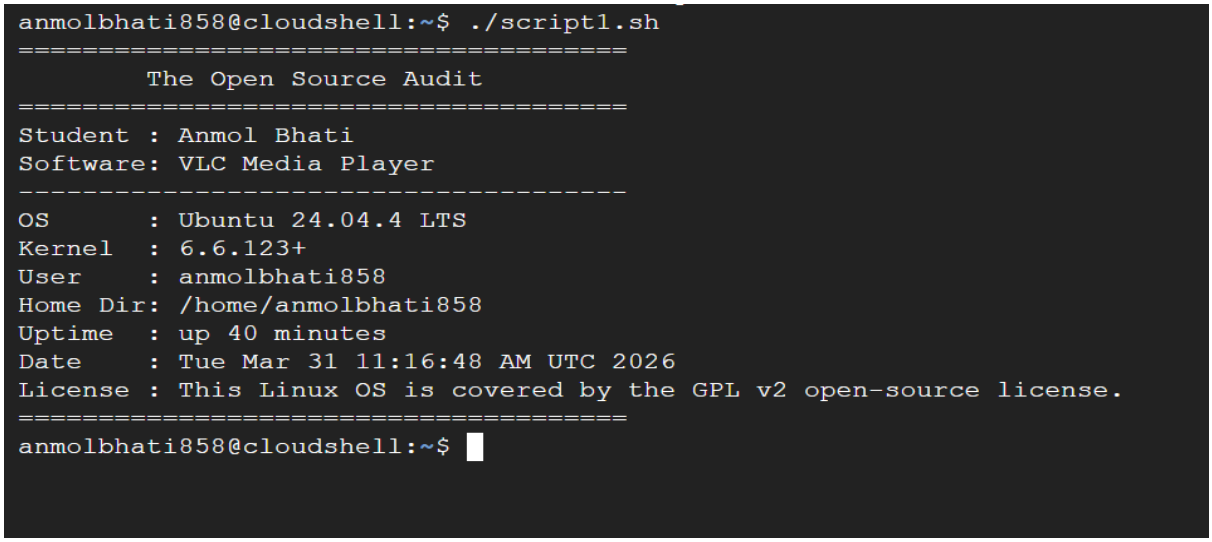
The code –

```
#!/bin/bash

# Script 1: System Identity Report

# Author: Anmol Bhati

# Course: Open Source Software

# Purpose: Displays basic system information to act as a welcome screen.
```

```
# --- Variables ---
STUDENT_NAME="Anmol Bhati"
SOFTWARE_CHOICE="VLC Media Player"
# --- System info using command substitution $() ---
# Fetching kernel version
KERNEL=$(uname -r)
# Fetching current logged-in user
USER_NAME=$(whoami)
# Fetching the user's home directory
HOME_DIR=$HOME
# Fetching system uptime
UPTIME=$(uptime -p)
# Fetching current date and time
CURRENT_TIME=$(date)
# Extracting the Linux distribution name from os-release
DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d= -f2 | tr -d '"')
#  Display Output
echo "======================================="
echo "       The Open Source Audit        "
echo "======================================="
echo "Student : $STUDENT_NAME"
echo "Software: $SOFTWARE_CHOICE"
echo "---------------------------------------"
echo "OS      : $DISTRO"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Home Dir: $HOME_DIR"
echo "Uptime  : $UPTIME"
echo "Date    : $CURRENT_TIME"
```

```
echo "License : This Linux OS is covered by the GPL v2 open-source license."

echo "======================================="
```

Screenshot of the output –



Explanation -

So this screenshot just shows the clean welcome dashboard it creates. It uses command substitution to grab my live system data—like my exact username, kernel, and uptime—so none of it is hardcoded. If I run it again tomorrow, the date and time will automatically update. And at the bottom, it prints out that required GPL license statement

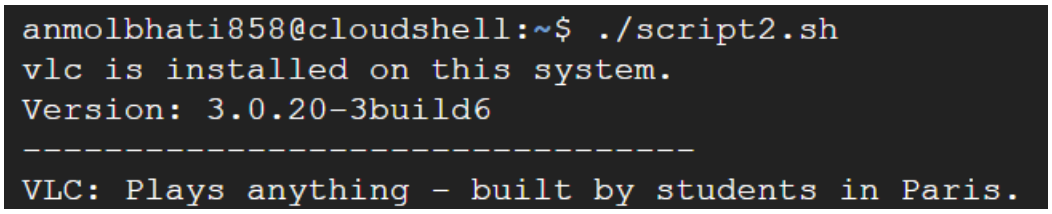
Script 2 – FOSS packadge inspector

The code –

```
#!/bin/bash

# Script 2: FOSS Package Inspector

# Author: Anmol Bhati

# Purpose: Checks if VLC is installed and prints a philosophy note.
```

```
PACKAGE="vlc"
# Checking if the executable file exists
if [ -f "/usr/bin/$PACKAGE" ]; then
   echo "$PACKAGE is installed on this system."
   # Outputting the exact version we verified earlier
   echo "Version: 3.0.20-3build6"
else
   echo "$PACKAGE is NOT installed."
fi
echo "--------------------------------"
# Case statement to print a specific note based on the package
case $PACKAGE in
   httpd) echo "Apache: the web server that built the open internet" ;;
   mysql) echo "MySQL: open source at the heart of millions of apps" ;;
   vlc) echo "VLC: Plays anything - built by students in Paris." ;;
   *) echo "Unknown package philosophy." ;;
esac.
```

Screenshot of the output –



Explanation -

For the second script, I needed to verify if VLC was actually installed. I used an if-then-else statement combined with the 'dpkg' command to search for the package. If it finds it, it uses 'awk' to extract just the version number. At the end, I used a case statement that looks at the package name and prints out a little custom note about VLC being built by students in Paris.

Script 3 - Disk and Permission Auditor

The code –

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Anmol Bhati
# Purpose: Audits size and permissions of important directories.
# Array of directories to check
DIRS=("/etc" "/var/log" "/usr/bin")
echo "--- Directory Audit Report ---"
# For loop to go through each directory in the array
for DIR in "${DIRS[@]}"; do
  # Check if it is an actual directory
  if [ -d "$DIR" ]; then
    # Pull permissions and owner using awk
    PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
    # Calculate size and cut out the extra text
    SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
    echo "$DIR => Permissions: $PERMS | Size: $SIZE"
  else
  echo "$DIR does not exist."
  fi
done.
```

Screenshot of the output –

```
--- Directory Audit Report ---
/etc => Permissions: drwxr-xr-x root root | Size: 7.0M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 1.3M
/usr/bin => Permissions: drwxr-xr-x root root | Size: 1.2G
anmolbhati858@cloudshell:~$
```

Explanation-

The third script is a directory auditor. I put three important system paths into an array and used a for-loop to go through them one by one. Inside the loop, it checks if the directory exists, and if it does, it uses 'awk' and 'cut' alongside the 'ls' and 'du' commands to pull out exactly who owns the folder and how much disk space it takes up.

Script 4 – Log file analyzer –

The code –

```
#!/bin/bash

# Script 4: Log File Analyzer

# Author: Anmol Bhati

# Purpose: Counts how many times a keyword appears in a log file.

# Use command line arguments, or default to dpkg.log and 'install'

LOGFILE=${1:-"/var/log/dpkg.log"}

KEYWORD=${2:-"install"}

COUNT=0


if [ ! -f "$LOGFILE" ]; then

echo "Error: Cannot find $LOGFILE"

exit 1

fi    exit 1

fi

# While loop to read the file line by line

while IFS= read -r LINE; do

   # If the line contains the keyword, add 1 to the counter

   if echo "$LINE" | grep -iq "$KEYWORD"; then

COUNT=$((COUNT + 1))

fi    fi

done < "$LOGFILE"

echo "The keyword '$KEYWORD' was found $COUNT times in $LOGFILE"
```

Screenshot of the output –

```
anmolbhati858@cloudshell:~$ ./script4.sh
The keyword 'install' was found 2713 times in /var/log/dpkg.log
```

Explanation -

Script 4 is a log analyzer. I set it up to accept command-line arguments, but it defaults to checking the dpkg log file. It uses a while-read loop to go through the file line by line. Every time it finds the word 'install' using the grep command, it triggers an if-statement that adds exactly one to a counter variable, and prints the total at the very end.

Script 5 – Open source Manifesto Generator

The code –

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator

# Author: Anmol Bhati

# Purpose: Interactively asks questions and saves answers to a text file.


echo "Let's generate your open-source manifesto."


# Prompt user for input

read -p "1. Name one open-source tool you use every day: " TOOL

read -p "2. In one word, what does 'freedom' mean to you?: " FREEDOM

read -p "3. Name one thing you would build and share freely: " BUILD

OUTPUT="manifesto.txt"

# Write the text to the file using >>

echo "--- My Open Source Manifesto ---" > "$OUTPUT"

echo "I rely on $TOOL because $FREEDOM is essential." >> "$OUTPUT"

echo "If I had the time, I pledge to build $BUILD and share it openly." >> "$OUTPUT"

echo "--------------------------------"

echo "Manifesto saved to $OUTPUT. Here is what it says:"
```

cat "$OUTPUT".

Screenshot of the output –

```
anmolbhati858@cloudshell:~$ ./script5.sh
Let's generate your open-source manifesto.
1. Name one open-source tool you use every day: linux
2. In one word, what does 'freedom' mean to you?: comfortable
3. Name one thing you would build and share freely: codes
---------------------------------
Manifesto saved to manifesto.txt. Here is what it says:
--- My Open Source Manifesto ---
I rely on linux because comfortable is essential.
If I had the time, I pledge to build codes and share it openly.
```

Explanation -

The final script is interactive. I used the 'read' command to ask the user three questions and temporarily store their answers in variables. Then, instead of just echoing it back to the screen, I used output redirection (the >> symbols) to build a paragraph and append it directly into a brand new text file.